

Documentație TSS:

1. Strategii de testare implementate:

În documentul *GradeManagementSystem_test.js* regăsim codul sursă al suitei de teste automate destinate verificării modulului *StudentManager* — componenta centrală a unui sistem de gestiune academică. Rolul acestui modul este simplu și bine definit: pe baza listei de note ale unui student, sistemul decide dacă acel student este eligibil pentru bursa de merit sau nu.

Logica de decizie se rezumă la două condiții care trebuie îndeplinite simultan: media aritmetică a tuturor notelor trebuie să fie cel puțin 8,50, iar lista de note nu trebuie să conțină nicio notă egală cu 4. Dacă ambele condiții sunt satisfăcute, sistemul returnează mesajul **"Scholarship Approved"**; în orice alt caz, returnează **"Scholarship Rejected"**.

Suita de teste acoperă exhaustiv toate scenariile posibile prin șase categorii distincte de testare: un test principal (testMain), testare de tip Black-Box prin Partiționare în Clase de Echivalență, Analiza Valorilor de Frontieră și Partiționare pe Categori, testare de tip White-Box prin Acoperirea Instrucțiunilor, Ramurilor, Condițiilor și Circuitelor, și în final Mutation Testing.

1.1. Testul Principal — testMain

Înainte de orice categorie specializată, suita conține un test de bază cu rol de verificare rapidă a scenariului fericit (happy path). Acesta confirmă că întregul lanț — creare obiect student, calcul medie, verificare condiții, returnare decizie — funcționează corect de la capăt la capăt.

testMain — Scenariul de bază

Testul verifică dacă un student cu toate notele peste medie și fără nota 4 primește bursa. Se creează un obiect **StudentManager** cu numele "Alice" și cu notele **[9, 9, 8, 9, 9]**. Media acestor note este $(9+9+8+9+9)/5 = 8,8$, care este mai mare decât pragul minim de 8,50. În plus, lista nu conține nota 4. Prin urmare, ambele condiții de eligibilitate sunt îndeplinite, iar sistemul ar trebui să returneze mesajul **"Scholarship Approved"**.

Acest test servește drept ancoră de sanitate: dacă eșuează, înseamnă că funcționalitatea de bază este complet compromisă, iar celelalte teste nu mai au relevanță.

1.2. Testare de Tip Black-Box

Testarea de tip Black-Box tratează modulul testat ca pe o cutie neagră: nu se cunoaște și nu se analizează implementarea internă — se cunosc doar intrările posibile și ieșirile așteptate. Scopul este să se verifice că sistemul se comportă corect pentru orice combinație de date de intrare, fără a privi în interior.

1.2.1 Partiționare în Clase de Echivalență (equivalencePartitioning)

Tehnica partiționării în clase de echivalență se bazează pe ideea că datele de intrare pot fi grupate în categorii (clase) cu comportament identic din perspectiva sistemului. Dacă un test dintr-o clasă trece sau pică, se presupune că același comportament se va manifesta pentru orice altă valoare din aceeași clasă. Astfel, în loc să testăm mii de combinații, testăm câte un reprezentant din fiecare clasă.

Sunt identificate șase clase de echivalență:

EP1 — Clasă validă, medie $\geq 8,50$, fără nota 4 → bursă aprobată.

Se creează studentul "Alice" cu notele [9, 9, 9]. Media este $(9+9+9)/3 = 9,0$ — mai mare decât 8,50 — și lista nu conține nota 4. Aceasta este clasa reprezentativă a tuturor cazurilor în care bursa trebuie acordată. Rezultatul așteptat este "Scholarship Approved".

EP2 — Clasă invalidă, notă în afara intervalului [1, 10] → excepție.

Se creează studentul "Bob" cu notele [9, 0, 9]. Nota 0 este în afara intervalului valid. Sistemul nu poate calcula o medie cu date corupte, deci trebuie să arunce o excepție cu mesajul exact "Grade must be a number between 1 and 10". Orice notă mai mică decât 1 sau mai mare decât 10 (indiferent că e 0, -5, 11 sau 99) face parte din această clasă invalidă.

EP3 — Clasă validă, medie sub 8,50 → bursă respinsă.

Se creează studenta "Carol" cu notele [6, 7, 8]. Media este $(6+7+8)/3 = 7,0$ — sub pragul de 8,50. Prima condiție de eligibilitate nu este îndeplinită, deci bursa se respinge indiferent de prezența sau absența notei 4. Rezultatul așteptat este "Scholarship Rejected".

EP4 — Clasă validă, medie $\geq 8,50$, dar conține nota 4 → bursă respinsă.

Se creează studentul "Dan" cu notele **[10, 10, 10, 4]**. Media este $(10+10+10+4)/4 = 8,5$ — suficient pentru prima condiție — însă nota 4 este prezentă și blochează acordarea bursei. Această clasă acoperă toți studenții cu medie suficientă dar cu nota 4 în catalog. Rezultatul așteptat este **"Scholarship Rejected"**.

EP5 — Clasă invalidă, lista de note este goală → excepție.

Se creează studenta "Eve" cu lista vidă **[]**. Nu există note pe baza cărora să se calculeze o medie, deci sistemul aruncă excepția **"The grade list is empty or invalid"**.

EP6 — Clasă invalidă, lista de note nu este un array (null) → excepție.

Se creează studentul "Frank" cu valoarea **null** în locul listei de note. Intrarea nu respectă tipul așteptat, deci sistemul aruncă același mesaj **"The grade list is empty or invalid"**. Această clasă cuprinde toate valorile non-array: **null**, **undefined**, șiruri de caractere, numere întregi etc.

1.2.2 Analiza Valorilor de Frontieră (boundaryValueAnalysis)

Analiza valorilor de frontieră completează partiționarea în clase de echivalență concentrându-se pe valorile extreme ale fiecărui interval. Experiența arată că erorile de programare apar cel mai frecvent exact la granițele intervalelor (off-by-one errors), de aceea se testează valorile de la limita inferioară, imediat sub ea, la limita superioară și imediat deasupra ei.

Frontierele identificate sunt: intervalul valid al notelor [1, 10] și pragul mediei de 8,50.

Frontiera inferioară a notelor individuale — intervalul valid [1, 10]:

Valoare testată	Poziție față de frontieră	Note de intrare	Rezultat așteptat
Nota 0	sub limita minimă	[0]	excepție
Nota 1	exact la limita minimă	[1]	"Scholarship Rejected"
Nota 2	imediat deasupra minimului	[2]	"Scholarship Rejected"

Nota 9	imediat sub maximum	[9]	"Scholarship Approved"
Nota 10	exact la limita maximă	[10]	"Scholarship Approved"
Nota 11	peste limita maximă	[11]	excepție

Descrierea detaliată a fiecărui test de frontieră:

Nota 0 (sub limita minimă): Nota 0 se află în afara intervalului valid. Sistemul o respinge înainte de orice calcul.

Nota 1 (exact la limita minimă): Nota 1 este prima valoare validă. Media este 1,0 — sub prag — deci bursa se respinge, fără excepție.

Nota 2 (imediat deasupra minimului): Nota 2 este validă. Media 2,0 este sub prag, deci respingere.

Nota 9 (imediat sub maximum): Nota 9 este validă. Media 9,0 depășește pragul, deci aprobare.

Nota 10 (exact la limita maximă): Nota 10 este ultima valoare validă. Media 10,0 duce la aprobare.

Nota 11 (peste limita maximă): Nota 11 depășește intervalul. Sistemul aruncă excepție.

Frontiera pragului mediei — 8,50:

Note [8, 9], medie 8,50 (exact la prag): Media $(8+9)/2 = 8,50$ este exact egală cu pragul. Condiția $\geq 8,50$ este satisfăcută (egalitate inclusă), deci bursa se acordă.

Note [8, 8, 9, 8], medie 8,25 (imediat sub prag): Media $(8+8+9+8)/4 = 8,25$ este sub 8,50. Condiția nu este îndeplinită, deci respingere.

Note [9, 9, 8], medie $\approx 8,67$ (imediat peste prag): Media $(9+9+8)/3 \approx 8,667$ depășește pragul. Condiția este satisfăcută, deci aprobare.

Frontiera valorii de blocare — nota 4:

Note [10,10,10,3,10], Nota 3: Nota 3 nu este valoarea de blocare. Media 8,6 depășește pragul și nu există nota 4, deci bursa se acordă.

Note [10,10,10,4], Nota 4: Nota 4 este exact valoarea de blocare. Chiar dacă media este 8,50 (la prag), prezența notei 4 respinge bursa.

Note [10,10,10,5,10], Nota 5: Nota 5 nu este valoarea de blocare. Media 9,0 depășește pragul și nu există nota 4, deci aprobare.

1.2.3 Partiționare pe Categori (categoryPartitioning)

Partiționarea pe categorii este o extensie a partiționării în clase de echivalență, care organizează datele de test în funcție de natura și tipul lor, nu doar de valorile numerice. Sunt definite cinci categorii:

Categoria A — Date de intrare invalide prin valoare: note în afara intervalului [1,10]. Se testează nota 0 și nota 11, ambele trebuind să producă excepția exactă "Grade must be a number between 1 and 10".

Categoria B — Date de intrare invalide prin tip: nota este un șir de caractere ("ten") sau lista nu este un array (undefined). Sistemul trebuie să recunoască tipul incorect și să arunce mesajul adecvat.

Categoria C — Medie sub prag: note valide dar cu medie sub 8,50. Decizia returnată trebuie să fie exact "Scholarship Rejected", fără excepție.

Categoria D — Medie peste prag, nota 4 prezentă: note valide cu medie $\geq 8,50$ dar care includ nota 4. Decizia returnată trebuie să fie "Scholarship Rejected" din cauza blocajului.

Categoria E — Toate condițiile pozitive: note valide, medie $\geq 8,50$, fără nota 4. Decizia returnată trebuie să fie exact "Scholarship Approved".

Suplimentar, sunt incluse două teste de negație: rezultatul aprobat nu trebuie să fie "Scholarship Rejected" (verifică absența unui string greșit), iar rezultatul respins nu trebuie să fie "Scholarship Approved". Aceste teste combat mutanții care inversează mesajele returnate.

1.3. Testare de Tip White-Box

Testarea de tip White-Box, spre deosebire de cea Black-Box, analizează structura internă a codului sursă. Scopul este să se asigure că fiecare instrucțiune, ramură și condiție din implementare este exercitată cel puțin o dată de suite de teste. Suita acoperă patru niveluri de acoperire (coverage).

1.3.1 Acoperirea Instrucțiunilor (statementCoverage)

La nivelul cel mai de bază, acoperirea instrucțiunilor garantează că fiecare linie de cod executabilă din funcția checkMeritScholarship a fost parcursă cel puțin o dată. Sunt identificate șase instrucțiuni distincte care trebuie acoperite:

SC1: Se transmite un șir de caractere ("invalid") în loc de array. Prima instrucțiune de gardă din funcție — care verifică dacă parametrul este un array — este activată și aruncă excepția "The grade list is empty or invalid".

SC2: Se transmite un array vid []. A doua instrucțiune de gardă — care verifică lungimea listei — este activată și aruncă aceeași excepție.

SC3: Se transmit notele [9, 9, 0, 8] — a treia notă este 0. Bucla de validare parcurge prima notă (validă), a doua (validă), iar la a treia activează instrucțiunea throw din validateGrade: "Grade must be a number between 1 and 10".

SC4: Se transmit notele [9, 9, 8] — toate valide, medie 8,67, fără nota 4. Bucla se finalizează fără erori, condiția finală este adevărată, iar instrucțiunea return "Scholarship Approved" este atinsă.

SC5: Se transmit notele [6, 7, 8] — toate valide, medie 7,0. Bucla se finalizează, condiția finală este falsă (medie < 8,50), iar instrucțiunea return "Scholarship Rejected" este atinsă pe ramura mediei mici.

SC6: Se transmit notele [10, 10, 10, 4] — medie 8,50 dar cu nota 4. Bucla se finalizează, condiția finală este falsă din cauza notei 4, iar instrucțiunea return "Scholarship Rejected" este atinsă pe ramura blocajului.

1.3.2 Acoperirea Ramurilor (branchCoverage)

Acoperirea ramurilor merge un pas mai departe: fiecare ramură a fiecărei instrucțiuni condiționale (if/else, throw, return) trebuie parcursă cel puțin o dată atât pe calea adevărată (true) cât și pe cea falsă (false). Sunt identificate patru ramuri principale:

B1-T (ramura adevărată din validateGrade): O notă sub minim (nota 0) și o notă peste maxim (nota 11) activează ramura de aruncare a excepției din funcția de validare. Ambele teste confirmă că această ramură funcționează.

B1-F (ramura falsă din validateGrade): O notă validă (nota 7) nu aruncă excepție — ramura de validare trece fără eroare.

B2-T și B3-T (gărzile de la începutul funcției): Un non-array ("invalid") activează prima gardă; un array vid ([]) activează a doua gardă. Ambele aruncă excepția "The grade list is empty or invalid".

B4-T și B4-F (condiția finală de decizie): Notele [9, 9, 8] activează ramura adevărată (aprobare). Notele [7, 8, 8] activează ramura falsă via medie mică, iar notele [10, 10, 10, 4] activează ramura falsă via prezența notei 4.

1.3.3 Acoperirea Condițiilor (conditionCoverage)

Acoperirea condițiilor verifică toate combinațiile posibile de valori (adevărat/fals) pentru fiecare condiție atomică din expresia finală de decizie. Condiția finală are forma: $\text{medie} \geq 8,50$ Și nu există nota 4. Denotând $C1 = (\text{medie} \geq 8,50)$ și $C2 = (\text{fără nota } 4)$, există patru combinații posibile:

Combinație	Note de intrare	Medie	Rezultat	Explicație
$C1 = \text{true}, C2 = \text{true}$	[9, 9, 9]	medie = 9,0	Approved	Singura combinație care duce la aprobare.
$C1 = \text{true}, C2 = \text{false}$	[10, 10, 10, 4]	medie = 8,5	Rejected	Media e suficientă, dar nota 4 blochează.
$C1 = \text{false}, C2 = \text{true}$	[7, 8, 8]	medie = 7,67	Rejected	Fără nota 4, dar media insuficientă.
$C1 = \text{false}, C2 = \text{false}$	[4, 7, 8]	medie = 6,33	Rejected	Ambele condiții sunt false simultan.

1.3.4 Acoperirea Circuitelor (circuitsCoverage)

Acoperirea circuitelor, numită și acoperirea căilor de execuție, urmărește toate traseele distincte prin care fluxul de control poate traversa funcția. Sunt identificate șase căi distincte:

Calea 1: *Parametrul grades nu este un array (undefined).* Excepție aruncată înainte de orice iterație a buclei: "The grade list is empty or invalid".

Calea 2: *Parametrul grades este un array vid [].* Excepție aruncată după verificarea tipului, înainte de buclă: "The grade list is empty or invalid".

Calea 3: *Array valid dar cu o notă invalidă la mijlocul listei: [9, 9, 0, 8].* Bucla pornește, parcurge primele două note valide, apoi la nota 0 aruncă excepție din interiorul buclei: "Grade must be a number between 1 and 10".

Calea 4: *Array valid, toate notele valide, medie < 8,50, fără nota 4: [6, 7, 8, 7].* Bucla se finalizează complet, condiția finală este falsă din cauza mediei mici, se returnează "Scholarship Rejected".

Calea 5: *Array valid, toate notele valide, medie $\geq 8,50$, nota 4 prezentă: [10, 10, 10, 4].* Bucla se finalizează, condiția finală este falsă din cauza notei 4, se returnează "Scholarship Rejected".

Calea 6: *Array valid, toate notele valide, medie $\geq 8,50$, fără nota 4: [9, 9, 8, 9].* Bucla se finalizează, ambele condiții sunt adevărate, se returnează "Scholarship Approved". Aceasta este singura cale care duce la aprobare.

Complexitatea Ciclomatică:

Complexitatea ciclomatică este o metrică software introdusă de Thomas J. McCabe în 1976, utilizată pentru a măsura complexitatea structurală a unui program. Aceasta cuantifică numărul de căi independente de execuție dintr-un modul, oferind astfel o estimare a efortului minim de testare necesar pentru acoperirea completă a codului. Cu cât valoarea este mai mare, cu atât codul este mai complex și mai dificil de testat și întreținut.

Complexitatea ciclomatică a modului StudentManager a fost calculată folosind formula $M = E - N + 2P$, unde E reprezintă numărul de muchii, N numărul de noduri, iar P numărul de componente conexe.

O metodă echivalentă și mai intuitivă este numărarea deciziilor logice din cod și adăugarea valorii 1:

- `if (!Array.isArray(grades) || grades.length === 0)` — contribuie cu 2 (un if + un operator ||)
- `for (let i = 0; i < grades.length; i++)` — contribuie cu 1
- `if (typeof grade !== 'number' || grade < 1 || grade > 10)` — contribuie cu 3 (un if + doi operatori ||)
- `if (average >= 8.50 && !grades.includes(4))` — contribuie cu 2 (un if + un operator &&)

$$M = 2 + 1 + 3 + 2 + 1 = 9$$

O complexitate ciclomatică de 9 indică faptul că există 9 căi independente de execuție în cadrul modului. Această valoare se reflectă direct în secțiunea `circuitsCoverage`, unde sunt identificate și testate 6 căi principale de execuție, restul fiind acoperite prin testele de tip Black-Box și Mutation Testing.

Valoarea obținută de 9 se încadrează în intervalul acceptabil (1, 10), ceea ce indică faptul că modulul StudentManager are o complexitate moderată, ușor de testat și de întreținut. Totodată, această valoare justifică rigoarea suitei de teste construite, care acoperă exhaustiv toate căile de execuție identificate.

1.4. Mutation Testing (killMutants)

Mutation Testing este o tehnică avansată de evaluare a calității suitei de teste. Ideea fundamentală este următoarea: dacă introducem intenționat o greșeală mică (un "mutant") în codul sursă — de exemplu, schimbăm `>=` cu `>` sau `&&` cu `||` — cel puțin un test din suită ar trebui să eșueze, "omorând" mutantul. Dacă niciun test nu detectează greșeala, suita este considerată slabă pentru acea zonă.

Sunt definite opt familii de mutanți și testele corespunzătoare care îi elimină:

M1 — Mutantul operatorului de comparație (`>=` → `>`): Dacă implementarea ar folosi `>` în loc de `>=` pentru compararea mediei cu pragul, un student cu media exact 8,50 ar fi incorect respins. Testul cu notele [8, 9] (medie exactă 8,50) verifică că rezultatul este Approved, omorând acest mutant. Complementar, testul cu media 8,25 confirmă că pragul nu a fost coborât accidental.

M2a/M2b — Mutanții de deplasare a pragului (`>= 8,51` sau `>= 8,0`): M2a verifică că media 8,50 este aprobată (elimină mutantul care ridică pragul la 8,51). M2b verifică că media 8,25 este respinsă (elimină mutanții care coboară pragul la 8,0 sau 8,49).

M3 — Mutantul de negație (! eliminat): Dacă operatorul de negație ar fi eliminat din condiția de verificare a notei 4, logica s-ar inversa: bursa ar fi acordată când nota 4 este prezentă și respinsă când lipsește. Testul verifică ambele sensuri simultan: [10,10,10,4] trebuie respins și [10,10,10,8] trebuie aprobat.

M4a/M4b/M4c — Mutanții valorii de blocare (`includes(4)` → `includes(3)` sau `includes(5)`): M4a confirmă că nota 3 nu blochează bursa (elimină mutantul care verifică `includes(3)`). M4b confirmă că nota 4 blochează bursa (confirmă valoarea corectă). M4c confirmă că nota 5 nu blochează bursa (elimină mutantul `includes(5)`).

M5a/M5b/M5c — Mutanții limitei minime a notelor (`< 1`): M5a: nota 1 este validă — elimină mutantul care coboară minimul la 0 (`grade < 0`). M5b: nota 0 este invalidă — elimină mutantul care ridică minimul (`grade < 0` → `grade < 1`). M5c: nota 2 este validă — elimină mutantul care ridică minimul la 2 (`grade < 2`).

M6a/M6b/M6c — Mutanții limitei maxime a notelor (`> 10`): M6a: nota 10 este validă — elimină mutantul care coboară maximul la 9 (`grade > 9`). M6b: nota 11 este invalidă — elimină mutantul care ridică maximul la 11 (`grade > 11`). M6c: nota 9 este validă — confirmă că maximul nu este setat greșit la 9.

M7a/M7b — Mutanții mesajelor returnate (mesaje inversate): M7a verifică că decizia de aprobare returnează exact "Scholarship Approved" și NU "Scholarship Rejected". M7b verifică opusul. Aceste teste detectează mutanții care inversează sau confundă cele două mesaje de răspuns.

M8a/M8b — Mutantul operatorului logic (&& → ||): Dacă && din condiția finală ar fi înlocuit cu ||, ar fi suficient ca o singură condiție să fie adevărată pentru aprobare — logică incorectă. M8a: medie >= 8,50 cu nota 4 trebuie să fie respins (nu suficientă media singură). M8b: medie < 8,50 fără nota 4 trebuie să fie respins (nu suficientă absența lui 4 singură).

Mutation Testing este o tehnică esențială deoarece evaluează calitatea suitei de teste în sine, nu doar a codului sursă. Un test care nu detectează niciun mutant nu oferă nicio garanție reală asupra corectitudinii implementării. În cazul modulului StudentManager, fiecare mutant definit reprezintă un tip concret de bug care ar putea apărea în producție: un operator greșit (>= înlocuit cu >), o valoare de prag modificată accidental (8.50 devine 8.51), o condiție logică inversată (&& devine ||) sau un mesaj returnat incorect. Dacă oricare dintre aceste greșeli ar scăpa nedetectată, sistemul ar putea acorda bursa unor studenți neeligibili sau ar putea respinge studenți care o merită, ceea ce în contextul unui sistem academic real ar reprezenta o eroare critică. Toate cele 16 teste din categoria killMutants au fost proiectate explicit pentru a detecta aceste scenarii, asigurând astfel că suita de teste este cu adevărat robustă și nu doar formal completă.

1.5. Cazul Special JavaScript — NaN (NaN edge case)

Suita include un test suplimentar care documentează un comportament specific limbajului JavaScript. Valoarea specială NaN (Not a Number) are proprietatea neașteptată că `typeof NaN === "number"` returnează `true` — deci o gardă de tipul "dacă nu este număr, aruncă excepție" nu o va detecta.

Testul creează studentul "Alice" cu notele **[NaN, 9, 9]**. Nota NaN trece de validarea de tip, dar corupte suma, transformând media în NaN. Expresia **NaN >= 8.50** returnează **false** în JavaScript, deci condiția de aprobare este falsă și sistemul returnează "Scholarship Rejected" — fără excepție, dar cu un rezultat care ar putea fi înșelător dacă nu este documentat. Testul nu afirmă că acesta este comportamentul corect, ci că acesta este comportamentul actual, pe care echipa îl cunoaște.

1.6. Rezumat și Statistici

Suita `GradeManagementSystem_test.js` conține în total 57 de teste individuale, organizate în 10 grupuri (describe blocks). Tabelul de mai jos rezumă distribuția:

Grup de teste	Nr. teste	Scop
testMain	1	Verificare end-to-end a scenariului fericit
equivalencePartitioning	6	Câte un reprezentant din fiecare clasă de echivalență
boundaryValueAnalysis	12	Valorile extreme ale notelor și ale pragului mediei
categoryPartitioning	7	Verificare per categorie de intrare, inclusiv negații
statementCoverage	6	Fiecare instrucțiune executabilă acoperită cel puțin o dată
branchCoverage	8	Fiecare ramură adevărată/falsă acoperită
conditionCoverage	4	Toate combinațiile C1×C2 din condiția finală
circuitsCoverage	6	Toate traseele distincte de execuție
killMutants	16	Eliminarea tuturor mutantilor identificați
NaN edge case	1	Comportament specific JavaScript documentat
TOTAL	57	

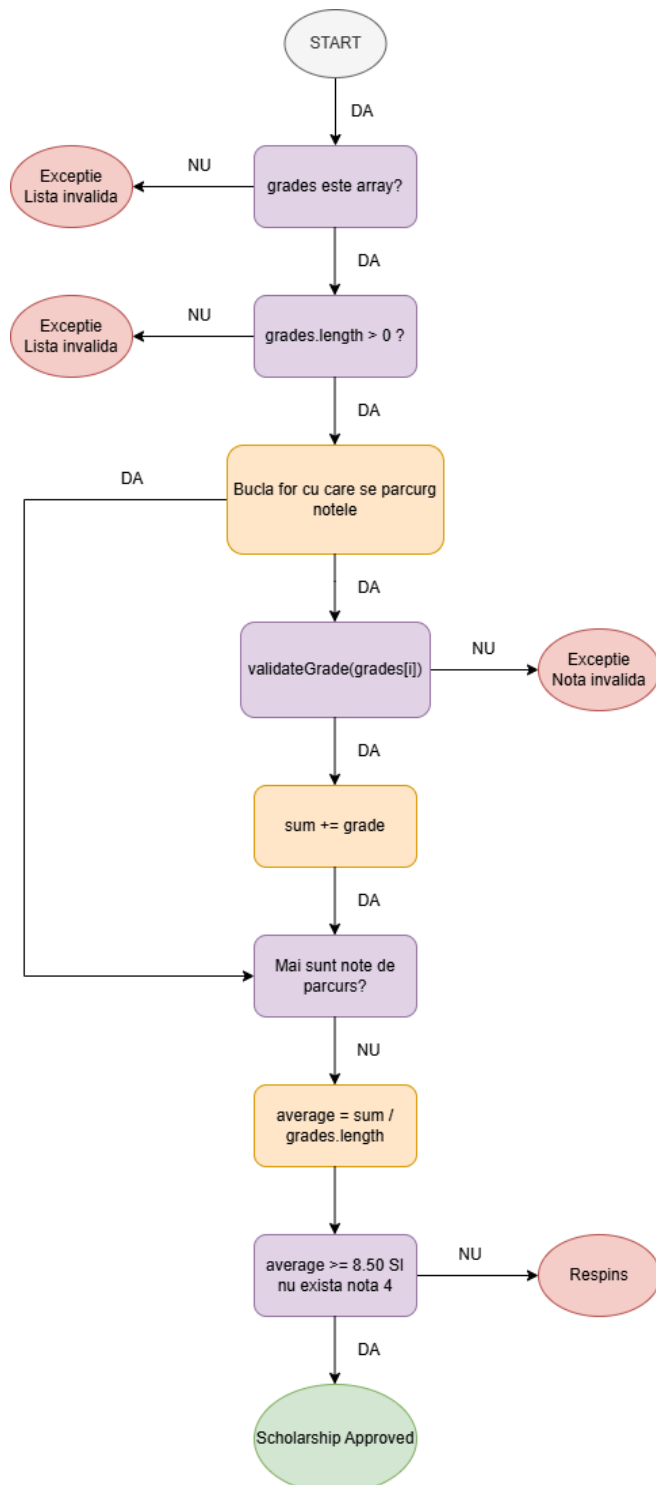
Prin combinarea tuturor celor șase tehnici de testare, suita acoperă exhaustiv atât comportamentul extern (Black-Box) cât și structura internă (White-Box) a modului StudentManager, oferind un grad ridicat de încredere că implementarea este corectă, robustă și rezistentă la modificări neintenționate.

2. [StudentManager.js](#):

În documentul **StudentManager.js** regăsim codul sursă al modului principal al aplicației — clasa **StudentManager** — care implementează logica de gestionare a notelor unui student și de decizie privind eligibilitatea pentru bursa de merit.

Clasa expune trei componente: constructorul, care primește numele studentului și lista sa de note și le stochează intern; funcția **validateGrade**, care verifică dacă o notă individuală respectă intervalul valid [1, 10] și aruncă o excepție dacă nu este număr sau dacă se află în afara acestui interval; și funcția principală **checkMeritScholarship**, care orchestrează întreaga logică de decizie.

Funcția `checkMeritScholarship` începe cu două verificări de gardă: dacă lista de note nu este un array sau dacă este vidă, se aruncă imediat excepția `"The grade list is empty or invalid"`. Dacă intrările sunt valide, funcția parcurge lista notă cu notă într-o buclă `for`, validând fiecare element prin apelul `validateGrade` și acumulând suma lor. La finalul buclei se calculează media aritmetică prin împărțirea sumei la numărul de note. Decizia finală se bazează pe două condiții evaluate simultan: media trebuie să fie mai mare sau egală cu `8.50` și lista nu trebuie să conțină nota `4`. Dacă ambele condiții sunt îndeplinite, funcția returnează `"Scholarship Approved"`; în orice alt caz returnează `"Scholarship Rejected"`.



3. Configurația sistemului:

ASUS VivoBook X512JP

Procesor: Intel Core i7-1065G7 @ 1.30GHz (1.50 GHz)

RAM: 8.00 GB (7.81 GB utilizabil)

Placă grafică: NVIDIA GeForce MX330 (2 GB) / Intel Iris Plus Graphics (128 MB)

Stocare: 477 GB (262 GB utilizat)

Sistem de operare: Windows 11

Runtime: [Node.js](#) v18.x sau mai nou

Framework Testare: Jest v30.3.0

Limbaj: JavaScript (ES6+)

IDEs folosite: VS Code

4. Interpretarea rezultatelor și cazuri speciale:

Un aspect interesant descoperit în timpul testării este **Edge Case-ul pentru NaN (Not-a-Number)**:

În JavaScript, `typeof NaN` este `'number'`. Testele demonstrează că o notă de tip NaN trece de prima validare, dar corupe media finală, ducând la respingerea bursei. Aceasta este o observație critică despre comportamentul limbajului.

5. Raport: Teste scrise de om vs. AI:

Pentru testul AI am folosit Gemini 1.5 Pro în scopul generării automate a unei suite de teste unitare pentru clasa [StudentManager.js](#).

Prompt-ul trimis către tool-ul de inteligență artificială a fost „Analizează clasa StudentManager.js și generează o suită de teste unitare folosind framework-ul Jest. Asigură-te că testezi calculul mediei și validările pentru note.”

Răspunsul a fost:

```
JavaScript

// Cod generat de AI
const StudentManager = require('./StudentManager');

describe('AI Generated Tests', () => {
  test('should calculate average correctly', () => {
    const sm = new StudentManager('Test', [10, 8]);
    expect(sm.checkMeritScholarship()).toBe('Scholarship Approved');
  });

  test('should reject if average is low', () => {
    const sm = new StudentManager('Test', [5, 6]);
    expect(sm.checkMeritScholarship()).toBe('Scholarship Rejected');
  });

  test('should throw error for invalid grade', () => {
    const sm = new StudentManager('Test', [11]);
    expect(() => sm.checkMeritScholarship()).toThrow();
  });
});
```

Caracteristică	Om	AI
Metodologie	Urmează riguros EP, BVA și Mutation Testing	Se bazează pe cazuri de utilizare comune (Happy Path)
Acoperirea ramurilor	100% (testează separat condițiile complexe)	~70% (ignoră cazul unui student cu media mare, dar cu o notă de 4)
Edge Cases	Include cazuri specifice de tip NaN, null sau undefined	Testează doar valori numerice standard
Documentare	Explică legătura cu teoria testării	Oferă doar descrieri scurte ale testelor

Așadar, concluziile sunt:

- Punctele forte ale AI-ul sunt viteza de execuție și ușurința în a genera rapid un „boiler plate” (schelet) pentru teste.
- Punctele sale slabe sunt ratarea condiției logice „invizibilă” (nota 4 blochează bursa chiar dacă media e mare) până când nu a fost instruit special. De asemenea, nu a considerat cazurile de „Boundary Value” (exact 8.50).
- Concluzie: AI-ul este un asistent bun pentru productivitate, dar nu poate înlocui expertiza umană în testarea de precizie (White-Box).